

Laporan Tugas Besar 1

IF2211 Strategi Algoritma

PEMANFAATAN ALGORITMA GREEDY DALAM PEMBUATAN BOT PERMAINAN BATTLECODE 2025

Semester II Tahun 2025/2026



Oleh kelompok
Stima Impressed

Faiq Azzam Nafidz (13524003)

Raysha Erviandika Putra (13524050)

Zeki Amani (13524082)

LABORATORIUM ILMU DAN REKAYASA KOMPUTASI
PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

1 Deskripsi Tugas

Battlecode adalah permainan strategi waktu nyata di mana Anda akan menulis kode untuk pemain otonom. Pemain Anda harus mengelola pasukan robot secara strategis dan mengendalikan cara robot-robot Anda bekerja sama untuk mengalahkan tim lawan. Sebagai peserta, Anda akan belajar menggunakan kecerdasan buatan, penelusuran jalur, algoritma terdistribusi, dan komunikasi untuk membuat pemain Anda bersaing mungkin. Namun, Anda hanya memiliki sumber daya komputasi terbatas per giliran, jadi Anda harus merencanakan strategi, mengoptimalkan, dan berkompromi sesuai kebutuhan. Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan strategi greedy dalam membuat bot ini.

2 Landasan Teori

2.1 Algoritma *Greedy*

Algoritma *greedy* merupakan metode sederhana yang populer untuk memecahkan persoalan optimasi. Seperti namanya, "*greedy*", yang berarti rakus, algoritma ini memiliki membuat solusi dengan cara mengambil keputusan terbaik dari semua pilihan pada tiap langkahnya tanpa mempertimbangkan konsekuensinya. Dengan kata lain, algoritma ini mencari optimum local pada tiap langkahnya dengan harapan bahwa langkah sisanya mengarah ke solusi optimum global.

Algoritma *greedy* tidak selalu menjamin solusi optimal. Jika tidak optimal, maka solusinya merupakan solusi sub-optimum atau pseudo-optimum. hal ini dikarenakan algoritma ini tidak mengevaluasi secara menyeluruh terhadap semua solusi yang ada dan Ada kemungkinan terdapat beberapa fungsi seleksi yang berbeda, sehingga pemilihan fungsi seleksi yang tidak tepat dapat menghasilkan solusi yang tidak optimal. Namun, algoritma ini tetap dapat sangat membantu mempercepat pencarian solusi dibandingkan brute force.

Greedy memiliki beberapa komponen yaitu,

1. **Himpunan kandidat**, C : berisi kandidat yang akan dipilih pada setiap langkah (misal: simpul/sisi di dalam graf, job, task, koin, benda, karakter, dsb).
2. **Himpunan solusi**, S : berisi kandidat yang sudah dipilih.
3. **Fungsi solusi** : menentukan apakah solusi sudah lengkap.
4. **Fungsi seleksi (selection function)** : memilih kandidat berdasarkan strategi greedy tertentu. Strategi greedy ini bersifat heuristik.
5. **Fungsi kelayakan (feasible)** : memeriksa apakah kandidat yang dipilih dapat dimasukkan ke dalam himpunan solusi (layak atau tidak).
6. **Fungsi obyektif** : memaksimumkan atau meminimumkan nilai tertentu.

Dengan menggunakan komponen di atas, maka dapat dikatakan bahwa, algoritma greedy melibatkan pencarian sebuah himpunan bagian, S , dari himpunan kandidat, C ; yang dalam hal ini, S harus memenuhi beberapa kriteria yang ditentukan, yaitu S menyatakan suatu solusi dan S dioptimisasi oleh fungsi objektif

2.2 Bot

Bot adalah program atau perangkat lunak yang bekerja secara otomatis untuk melakukan suatu tugas berulang. Dalam konteks Tugas Besar ini, bot yang kami buat digunakan untuk menyelesaikan game ini. Bot akan mengeksekusi sebuah aksi tiap rondennya, aksi tersebut bisa dilakukan oleh salah satu tower, soldier, splasher, ataupun mopper yang ada di map bergantung rondennya namun tetap dikendalikan oleh instruksi program yang sama.

Diawal kita diberikan sebuah template berupa sebuah kelas yang akan digunakan untuk pembuatan bot, isinya sebagai berikut,

```

package examplefuncsplayer;

import battlecode.common.*;

import java.util.Arrays;
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Random;
import java.util.Set;

/**
 * RobotPlayer is the class that describes your main robot strategy.
 * The run() method inside this class is like your main function: this is what we'll call
 *   once your robot
 * is created!
 */
public class RobotPlayer {
    /**
     * We will use this variable to count the number of turns this robot has been alive.
     * You can use static variables like this to save any information you want. Keep in mind
     *   that even though
     * these variables are static, in Battlecode they aren't actually shared between your
     *   robots.
     */
    static int turnCount = 0;

    /**
     * A random number generator.
     * We will use this RNG to make some random moves. The Random class is provided by the
     *   java.util.Random
     * import at the top of this file. Here, we *seed* the RNG with a constant number (6147);
     *   this makes sure
     * we get the same sequence of numbers every time this code is run. This is very useful
     *   for debugging!
     */
    static final Random rng = new Random(6147);

    /** Array containing all the possible movement directions. */
    static final Direction[] directions = {
        Direction.NORTH,
        Direction.NORTHEAST,
        Direction.EAST,
        Direction.SOUTHEAST,
        Direction.SOUTH,
        Direction.SOUTHWEST,
        Direction.WEST,
        Direction.NORTHWEST,
    };

    /**
     * run() is the method that is called when a robot is instantiated in the Battlecode
     *   world.
     * It is like the main function for your robot. If this method returns, the robot dies!
     *
     * @param rc The RobotController object. You use it to perform actions from this robot,
     *   and to get
     *   information on its current status. Essentially your portal to interacting
     *   with the world.
     */
}

```

```

@SuppressWarnings("unused")
public static void run(RobotController rc) throws GameActionException {
    // Hello world! Standard output is very useful for debugging.
    // Everything you say here will be directly viewable in your terminal when you run a
    // match!
    System.out.println("I'm alive");

    // You can also use indicators to save debug notes in replays.
    rc.setIndicatorString("Hello world!");

    while (true) {
        // This code runs during the entire lifespan of the robot, which is why it is in
        // an infinite
        // loop. If we ever leave this loop and return from run(), the robot dies! At the
        // end of the
        // loop, we call Clock.yield(), signifying that we've done everything we want to
        // do.

        turnCount += 1; // We have now been alive for one more turn!

        // Try/catch blocks stop unhandled exceptions, which cause your robot to explode.
        try {
            // The same run() function is called for every robot on your team, even if
            // they are
            // different types. Here, we separate the control depending on the UnitType,
            // so we can
            // use different strategies on different robots. If you wish, you are free to
            // rewrite
            // this into a different control structure!
            switch (rc.getType()){
                case SOLDIER: runSoldier(rc); break;
                case MOPPER: runMopper(rc); break;
                case SPLASHER: break; // Consider upgrading examplefuncsplayer to use
                // splashers!
                default: runTower(rc); break;
            }
        }
        catch (GameActionException e) {
            // Oh no! It looks like we did something illegal in the Battlecode world. You
            // should
            // handle GameActionExceptions judiciously, in case unexpected events occur in
            // the game
            // world. Remember, uncaught exceptions cause your robot to explode!
            System.out.println("GameActionException");
            e.printStackTrace();

        }
        catch (Exception e) {
            // Oh no! It looks like our code tried to do something bad. This isn't a
            // GameActionException, so it's more likely to be a bug in our code.
            System.out.println("Exception");
            e.printStackTrace();

        }
        finally {
            // Signify we've done everything we want to do, thereby ending our turn.
            // This will make our code wait until the next turn, and then perform this
            // loop again.
            Clock.yield();
        }
        // End of loop: go back to the top. Clock.yield() has ended, so it's time for
        // another turn!
    }
}

```

```

        // Your code should never reach here (unless it's intentional)! Self-destruction
        imminent...
    }

/**
 * Run a single turn for towers.
 * This code is wrapped inside the infinite loop in run(), so it is called once per turn.
 */
public static void runTower(RobotController rc) throws GameActionException{
    // Pick a direction to build in.
    Direction dir = directions[rng.nextInt(directions.length)];
    MapLocation nextLoc = rc.getLocation().add(dir);
    // Pick a random robot type to build.
    int robotType = rng.nextInt(3);
    if (robotType == 0 && rc.canBuildRobot(UnitType.SOLDIER, nextLoc)){
        rc.buildRobot(UnitType.SOLDIER, nextLoc);
        System.out.println("BUILT A SOLDIER");
    }
    else if (robotType == 1 && rc.canBuildRobot(UnitType.MOPPER, nextLoc)){
        rc.buildRobot(UnitType.MOPPER, nextLoc);
        System.out.println("BUILT A MOPPER");
    }
    else if (robotType == 2 && rc.canBuildRobot(UnitType.SPLASHER, nextLoc)){
        // rc.buildRobot(UnitType.SPLASHER, nextLoc);
        // System.out.println("BUILT A SPLASHER");
        rc.setIndicatorString("SPLASHER NOT IMPLEMENTED YET");
    }
}

// Read incoming messages
Message[] messages = rc.readMessages(-1);
for (Message m : messages) {
    System.out.println("Tower received message: '#' + m.getSenderID() + " " +
        m.getBytes());
}

// TODO: can we attack other bots?
}

/**
 * Run a single turn for a Soldier.
 * This code is wrapped inside the infinite loop in run(), so it is called once per turn.
 */
public static void runSoldier(RobotController rc) throws GameActionException{
    // Sense information about all visible nearby tiles.
    MapInfo[] nearbyTiles = rc.senseNearbyMapInfos();
    // Search for a nearby ruin to complete.
    MapInfo curRuin = null;
    for (MapInfo tile : nearbyTiles){
        if (tile.hasRuin()){
            curRuin = tile;
        }
    }
    if (curRuin != null){
        MapLocation targetLoc = curRuin.getMapLocation();
        Direction dir = rc.getLocation().directionTo(targetLoc);
        if (rc.canMove(dir))
            rc.move(dir);
        // Mark the pattern we need to draw to build a tower here if we haven't already.
        MapLocation shouldBeMarked = curRuin.getMapLocation().subtract(dir);
        if (rc.senseMapInfo(shouldBeMarked).getMark() == PaintType.EMPTY &&
            rc.canMarkTowerPattern(UnitType.LEVEL_ONE_PAINT_TOWER, targetLoc)){

```

```

        rc.markTowerPattern(UnitType.LEVEL_ONE_PAINT_TOWER, targetLoc);
        System.out.println("Trying to build a tower at " + targetLoc);
    }
    // Fill in any spots in the pattern with the appropriate paint.
    for (MapInfo patternTile : rc.senseNearbyMapInfos(targetLoc, 8)){
        if (patternTile.getMark() != patternTile.getPaint() && patternTile.getMark()
            != PaintType.EMPTY){
            boolean useSecondaryColor = patternTile.getMark() ==
                PaintType.ALLY_SECONDARY;
            if (rc.canAttack(patternTile.getMapLocation()))
                rc.attack(patternTile.getMapLocation(), useSecondaryColor);
        }
    }
    // Complete the ruin if we can.
    if (rc.canCompleteTowerPattern(UnitType.LEVEL_ONE_PAINT_TOWER, targetLoc)){
        rc.completeTowerPattern(UnitType.LEVEL_ONE_PAINT_TOWER, targetLoc);
        rc.setTimelineMarker("Tower built", 0, 255, 0);
        System.out.println("Built a tower at " + targetLoc + "!");
    }
}

// Move and attack randomly if no objective.
Direction dir = directions[rng.nextInt(directions.length)];
MapLocation nextLoc = rc.getLocation().add(dir);
if (rc.canMove(dir)){
    rc.move(dir);
}
// Try to paint beneath us as we walk to avoid paint penalties.
// Avoiding wasting paint by re-painting our own tiles.
MapInfo currentTile = rc.senseMapInfo(rc.getLocation());
if (!currentTile.getPaint().isAlly() && rc.canAttack(rc.getLocation())){
    rc.attack(rc.getLocation());
}
}

/**
 * Run a single turn for a Mopper.
 * This code is wrapped inside the infinite loop in run(), so it is called once per turn.
 */
public static void runMopper(RobotController rc) throws GameActionException{
    // Move and attack randomly.
    Direction dir = directions[rng.nextInt(directions.length)];
    MapLocation nextLoc = rc.getLocation().add(dir);
    if (rc.canMove(dir)){
        rc.move(dir);
    }
    if (rc.canMopSwing(dir)){
        rc.mopSwing(dir);
        System.out.println("Mop Swing! Booyah!");
    }
    else if (rc.canAttack(nextLoc)){
        rc.attack(nextLoc);
    }
    // We can also move our code into different methods or classes to better organize it!
    updateEnemyRobots(rc);
}

public static void updateEnemyRobots(RobotController rc) throws GameActionException{
    // Sensing methods can be passed in a radius of -1 to automatically
    // use the largest possible value.
    RobotInfo[] enemyRobots = rc.senseNearbyRobots(-1, rc.getTeam().opponent());

```


3.1.4 Fungsi Seleksi

Fungsi ini yang menentukan strategi dari masing masing bot, dengan pendekatan masing-masing bot sebagai berikut.

1. Fokus menyelesaikan objektif yang ditentukan.
2. Fokus memperbanyak resource.
3. Fokus memperkuat defense.

3.1.5 Fungsi Kelayakan

- Cadangan cat robot > 0
- *Movement cooldown* < 10 untuk aksi bergerak
- *Action cooldown* < 10 untuk aksi serang, warnai, spawn, transfer, atau withdraw
- Petak tujuan tidak diisi robot atau menara lain
- Petak tujuan bukan tembok (*wall*) atau reruntuhan (*ruins*)
- Petak target berada dalam radius aksi robot (radius 3 *soldier*, 2 *splasher*, $\sqrt{2}$ *mopper*)
- Petak target dapat dilihat robot (dalam jarak pandang $\sqrt{20}$)
- Cat robot/menara mencukupi biaya aksi yang dipilih
- *Chips* tim mencukupi biaya spawn atau pembangunan menara
- Untuk spawn: petak tujuan kosong dan dalam radius $\sqrt{4}$ dari menara, serta jumlah menara tim < 25
- Untuk membangun menara: lokasi adalah reruntuhan yang belum ada menaranya dan pola cat 5×5 sudah sesuai
- Untuk transfer/withdraw: target sekutu berada dalam jarak $\sqrt{2}$
- Untuk komunikasi: robot terhubung ke menara melalui jalur cat sekutu dan belum mengirim pesan di ronde ini
- Bytecode yang tersisa masih mencukupi untuk menjalankan aksi (batas 17,500 robot, 20,000 menara)

3.1.6 Fungsi Objektif

Fungsi objektif dari permasalahan ini adalah memaksimalkan jumlah petak yang diwarnai pada peta, sesuai dengan ketentuan pemenang dari permasalahan ini.

3.2 Alternatif Solusi

3.2.1 Impressive General

Strategi pada alternatif robot ini mengedepankan penyelesaian objektif khusus untuk tiap tipe robot. Robot *soldier* memiliki objektif membangun *tower*, *mopper* menyerang robot lawan serta mendistribusikan cat, *splasher* mengecat sebanyak-banyaknya petak lawan ataupun petak kosong, serta tower membangun robot sebanyak-banyaknya dengan tipe robot sesuai dengan keadaan di sekitarnya. Jika suatu robot bisa melakukan objektifnya dalam suatu waktu, maka objektif tersebut dilakukan terlebih dahulu sebelum melakukan hal lainnya.

Jika robot tidak bisa melakukan objektifnya, robot akan melakukan pergerakan dan penyerangan/pengecatan yang arah dan targetnya masing-masing ditentukan dengan kalkulasi yang berbeda. Arah "terbaik" ditentukan dengan menghitung poin pada tiap petak dan poin tersebut diakumulasikan

sesuai dengan letak petak terhadap robot sehingga setiap arah memiliki poin yang berbeda-beda. Sebagai contoh, robot *soldier* akan memilih arah dengan petak kosong paling banyak untuk dicat sehingga arah tersebut harus menjadi arah yang memiliki poin terbanyak pada kalkulasi.

Perhitungan nilai berbeda-beda untuk setiap robot. Robot *soldier* lebih mengedepankan petak kosong dibandingkan petak sekutu maupun musuh; robot *splasher* dan *mopper* mengedepankan petak lawan serta petak yang ada musuhnya; sedangkan *tower* mengikuti perhitungan robot yang ingin dibuatnya. Setiap robot juga mencoba untuk saling menjauh untuk mendorong eksplorasi area peta yang belum dikuasai. Setiap robot juga lebih memilih untuk pergi ke petak-petak yang bisa dilalui (petak yang tidak bisa dilalui: petak di luar peta, petak tembok) untuk mencegah pengumpulan robot di ujung-ujung peta dan mendorong robot untuk bergerak ke bagian tengah peta. Dengan perhitungan ini, tiap pergerakan robot dapat menuju ke petak yang menurut penglihatannya (mengingat penglihatan setiap robot terbatas) paling memberikan peluang untuk memenangkan permainan, sesuai dengan algoritma *greedy* yang mencoba mengambil keputusan yang terbaik di tiap langkah.

Pembuatan robot oleh tiap *tower* juga dikalkulasikan untuk memberikan keputusan terbaik di setiap gilirannya. Terdapat kondisi-kondisi yang memengaruhi pembuatan robot oleh *tower*. Kondisi yang pertama adalah ketika di sekitar *tower* tidak ada atau sedikit robot sekutu. Kondisi ini menyebabkan *tower* mengutamakan membuat robot *soldier* untuk melakukan eksplorasi dan membangun lebih banyak *tower* lain. Kondisi kedua adalah ketika sudah banyak robot di dekatnya atau terdapat komponen musuh (robot musuh atau petak yang dicat musuh) di sekitarnya. Pada kondisi kedua ini, *tower* akan mengutamakan membuat robot *mopper* dan *splasher* untuk menguasai area dan mempertahankan *tower* dari serangan musuh.

1. **Analisis Efisiensi:** Pendekatan ini cukup kompleks diimplementasikan karena setiap robot memiliki objektif yang berbeda-beda di waktu yang berbeda-beda juga. Pengimplementasian untuk tiap objektif cukup sederhana, hanya memerlukan pengecekan kondisi (*if-else*) atau pengulangan yang sering digunakan untuk pengecekan setiap petak di sekitar robot (*loop*),
2. **Analisis Efektivitas:** Walau implementasi cukup kompleks, pendekatan ini cukup efektif dalam menguasai area-area penting dalam peta. Dengan *tower* yang membuat robot sebanyak-banyaknya, *soldier* yang fokus membangun *tower*, serta *splasher* dan *mopper* yang melindungi area dalam peta membuat pendekatan ini efektif memenangkan permainan di peta yang memiliki tujuan-tujuan yang jelas (seperti terdapat *ruin* yang bisa dibuat *tower* di dekat *tower* awal dan tidak ada tembok penghalang antar *ruin* di peta). Namun, strategi ini kurang baik ketika permainan dijalankan di peta yang memiliki banyak tembok penghalang. Karena perhitungan arah "terbaik" yang dijelaskan sebelumnya, robot-robot yang dibuat tidak dengan sendirinya bergerak melewati celah-celah tembok yang ada di sekitarnya dan berusaha menjauhinya karena menganggap bergerak ke arah petak tembok bukan pilihan terbaik untuk saat itu.

3.2.2 Mr Capitalist

Strategi bot ini berfokus pada objektif yang paling penting terlebih dahulu, yaitu membangun dan melakukan upgrade tower agar chip dan paint cepat terkumpul. Tower yang digunakan hanya money tower dan paint tower. Soldier bertugas mencari ruin dan menyelesaikan pola tower, lalu ketika jumlah tower sudah cukup, soldier juga akan mulai menyelesaikan resource pattern (SRP). Sementara itu, mopper digunakan untuk membersihkan paint musuh dan menekan unit lawan di sekitarnya, sedangkan splasher digunakan untuk mengecat area yang memberikan hasil paling besar.

Jika tidak ada objektif utama yang bisa segera dikerjakan, bot menggunakan pendekatan *greedy* untuk memilih aksi terbaik pada giliran tersebut. Setiap unit menentukan target dan pergerakan berdasarkan kondisi lokal, seperti jarak ke tujuan, warna paint pada petak sekitar, dan keuntungan langsung dari aksi yang diambil.

1. **Analisis Efisiensi :** Strategi ini cukup efisien karena bot tidak melakukan pencarian global atau simulasi banyak langkah, tetapi hanya memanfaatkan informasi lokal pada setiap round. Pada program ini, soldier hanya memeriksa ruin terdekat, kandidat SRP di area sekitar, dan target eksplorasi lokal. Mopper memilih unit lawan atau petak musuh terdekat, dan splasher menilai titik serang yang ada di sekitarnya. Sedangkan fungsi movement hanya membandingkan 8 arah dengan skor sederhana berdasarkan jarak dan warna paint. Tower juga hanya mengecek musuh yang bisa diserang, kondisi upgrade, lalu memilih unit yang akan dibangun.

2. **Analisis Efektivitas** :Strategi ini efektif terutama pada map besar, karena bot berusaha membangun dan melakukan upgrade tower terlebih lalu amplifikasi melalui SRP, yang dimana sangat efektif di ruang terbuka. Jika banyak ruin berhasil dicapai dan dikonversi menjadi tower, maka bot dapat memastikan paint, dan chip seimbang sampai late game. Namun, efektivitasnya akan menurun apabila ekspansi awal gagal atau jumlah ruin yang dapat diamankan terlalu sedikit.

3.2.3 Minister Of Defense

strategi pada alternatif robot ini fokus mengevaluasi pilihan berdasarkan dampaknya terhadap kemampuan defensif, dengan kata lain greedy by defense. pada tiap ronde, bot akan memilih aksi yang dapat memperkuat defense seperti membangun defense tower, fokus menyerang musuh terlemah, dan robot fokus melindungi tower sebagai default behavior.

1. **Analisis Efisiensi** : Karena pada tiap round bot hanya mencari posisi tower sekutu, posisi ruin terdekat, dan posisi musuh terlemah terdekat, maka algoritma yang digunakan sangat simple yang membuatnya sangat efisien.
2. **Analisis Efektivitas** :strategi ini akan sangat bergantung terhadap apakah di awal kita bisa merebut sebanyak mungkin ruin. Jika berhasil, maka dengan menerapkan strategi defensif peluang kemenangan akan lumayan baik. Namun, jika gagal, strategi ini tentu sia-sia.

3.3 Pilihan Strategi Algoritma

Ketiga solusi memiliki kekurangan dan kelebihan masing masing, namun, dengan mempertimbangkan kemampuan untuk meraih kemenangan, General Impressive secara keseluruhan merupakan solusi terbaik. Hal ini karena pendekatan ini cukup efektif dalam menguasai area-area penting dalam peta banyak kondisi. berbeda dengan pendekatan lain yang dapat terhalang dalam beberapa kondisi, seperti Mr Capitalist yang kurang cocok untuk di map dimana mendapatkan tower sulit dan Minister Of Defense yang sangat bergantung ke hasil rng di awal permainan. Maka dari itu, solusi greedy yang kami pilih adalah General Impressive.

4 Implementasi dan Pengujian

4.1 Pseudocode

4.1.1 Impressive General

```
procedure run(rc):
  if rc.type == SOLDIER then runSoldier(rc)
  else if rc.type == MOPPER then runMopper(rc)
  else if rc.type == SPLASHER then runSplasher(rc)
  else runTower(rc)

procedure runTower(rc)
  enemyExist <- checkEnemyNearby(rc)
  if count(senseNearbyRobots(rc)) < (roundNumber(rc) / 500 + 2) AND enemyExist = false then
    robotToBuild  SOLDIER
  else
    if roundNumber(rc) mod 4 = 0 then
      robotToBuild  MOPPER
    else
      robotToBuild  SPLASHER

  dir <- bestDirection(rc)
  nextLoc <- location(rc) + dir

  if (enemyExist AND paint(rc) > paintCost(SPLASHER)) OR
    (canBuildRobot(rc, robotToBuild, nextLoc) AND
     chips(rc) > 1000 + moneyCost(SPLASHER) AND
```

```

    paint(rc) > paintCost(SPLASHER)) then

    buildRobot(rc, robotToBuild, nextLoc)

nearbyRobots <- senseNearbyRobots(rc, opponentTeam)
for each robot in nearbyRobots do
    if canAttack(rc, location(robot)) then
        attack(rc, location(robot))
    else
        break

procedure runSoldier(rc)
    if uselessCount < 5
        getPaint(rc)
        soldierObjective(rc)
    if isMovementReady(rc)
        dir <- bestDirection(rc)
        if canMove(rc, dir)
            move(rc, dir)
    if isActionReady(rc)
        attackNearbyTile(rc)

procedure runMopper(rc)
    getPaint(rc)
    mopperObjective(rc)
    if NOT isActionReady(rc) AND NOT isMovementReady(rc)
        return
    dir <- bestDirection(rc)
    if NOT enemyPaint(nextLoc)
        forceMove(nextLoc, rc)
    if canAttack(rc, location(rc)) AND enemyPaint(location(rc))
        attackNearbyEnemyPaint(rc)

procedure runSplasher(rc)
    getPaint(rc)
    dir <- bestDirection(rc)
    if canMove(rc, dir)
        move(rc, dir)
    if canAttack(rc, nextLoc)
        attackNearbyEnemyOrEmpty(rc)

procedure soldierObjective(rc)
    if numberTowers(rc) < 3 + roundNumber(rc) / 20 AND numberTowers(rc) < 5 AND
        numberTowers(rc) > 2
        towerToBuild <- LEVEL_ONE_MONEY_TOWER
    else
        towerToBuild <- LEVEL_ONE_PAINT_TOWER
    nearbyTiles <- senseNearbyMapInfos(rc)
    curRuin <- nearestUnbuiltRuin(rc, nearbyTiles)
    if existPaintTowerNearby(rc, nearbyTiles)
        towerToBuild <- LEVEL_ONE_MONEY_TOWER
    if curRuin != null
        targetLoc <- location(curRuin)
        if NOT patternFull(curRuin, rc)
            forceMove(targetLoc, rc)
            if markNeeded(rc, curRuin, towerToBuild)
                markTowerPattern(rc, towerToBuild, targetLoc)
            if isActionReady(rc)
                paintTowerPattern(rc, targetLoc)
        if canCompleteMoneyTower(rc, targetLoc)
            completeMoneyTower(rc, targetLoc)
        else if canCompletePaintTower(rc, targetLoc)

```

```

        completePaintTower(rc, targetLoc)
    upgradeNearbyTower(rc)

procedure mopperObjective(rc)
    if NOT isActionReady(rc)
        return
    target <- nearestEnemyRobot(rc)
    if target != null
        moveAndAttackEnemy(rc)
    target <- nearestFriendRobot(rc)
    if target != null
        moveAndGivePaint(rc)

procedure bestDirection(rc)
    paintScore <- array[8] initialized 0
    nearbyTiles <- senseNearbyMapInfos(rc)
    for each tile in nearbyTiles
        if isWall(tile)
            continue
        dir <- direction(rc, location(tile))
        if dir = CENTER
            continue
        index <- ordinal(dir)
        if rc.type = SOLDIER OR rc.type = TOWER AND robotToBuild = SOLDIER
            if paint(tile) = EMPTY
                paintScore[index] <- paintScore[index] + 100
            else if robotExists(tile)
                paintScore[index] <- paintScore[index] - 200
            else if allyPaint(tile)
                paintScore[index] <- paintScore[index] + 10
            else if enemyPaint(tile)
                paintScore[index] <- paintScore[index] + 5
        if rc.type = MOPPER OR rc.type = SPLASHER OR rc.type = TOWER AND (robotToBuild =
            MOPPER OR robotToBuild = SPLASHER)
            if enemyRobot(tile)
                paintScore[index] <- paintScore[index] + 100
            if enemyPaint(tile)
                paintScore[index] <- paintScore[index] + 100
            else if robotExists(tile)
                paintScore[index] <- paintScore[index] - 200
            else if paint(tile) = EMPTY
                paintScore[index] <- paintScore[index] + 10
            else if allyPaint(tile)
                paintScore[index] <- paintScore[index] + 5
    bestIndex <- 0
    lastDir <- direction(rc, lastLoc)
    if lastDir = directions[bestIndex]
        bestIndex <- bestIndex + 1
    bestLoc <- location(rc) + directions[bestIndex]
    for i <- bestIndex + 1 to 7
        if paintScore[i] >= paintScore[bestIndex] AND
            lastDir = directions[i] AND
            (canMove(rc, directions[i]) OR canBuild(rc, robotToBuild, bestLoc))
            bestIndex <- i
            bestLoc <- location(rc) + directions[bestIndex]
    return directions[bestIndex]

```

4.1.2 Mr Capitalist

```

procedure run(rc):

```

```

    if rc.type = SOLDIER then runSoldier(rc)
    else if rc.type = MOPPER then runMopper(rc)
    else if rc.type = SPLASHER then runSplasher(rc)
    else runTower(rc)

procedure runTower(rc):
    attackweakest()
    if not rc.isActionReady() then return
    if rc.canUpgradeTower(rc.getLocation()) then upgrade
    want <- chooseSpawnType(rc)
    rc.buildRobot(want, spawn)

procedure runSoldier(rc):
    updateTowerProgress(rc)
    if paint low then refill(rc)
    ruin <- nearestOpenRuin(rc)
    if ruin != null then doRuin(rc, ruin)
    if SRP condition valid then
        srp <- locateSRP(rc, rc.getLocation())
        if srp != null then doSRP(rc, srp)
    goal <- localExploreTarget(rc)
    if goal = null then goal <- laneTarget(rc)
    move(rc, goal)
    paintBest(rc)

procedure runMopper(rc):
    if paint low then refill(rc)
    enemy <- nearestEnemyUnit(rc)
    if enemy != null then attack / mopSwing / move(rc, enemy.location)
    dirty <- nearestEnemyPaintLoc(rc)
    if dirty != null then attack / move(rc, dirty)
    move(rc, laneTarget(rc))

procedure runSplasher(rc):
    if paint low then refill(rc)
    best <- best splash target
    if best != null then attack(best)
    else move(rc, enemyBase)

```

4.1.3 Minister Of Defense

```

procedure run(rc):
    if rc.type = SOLDIER then runSoldier(rc)
    if rc.type = TOWER then runTower(rc)
    if rc.type = SPLASHER then runSplasher(rc)
    if rc.type = MOPPER then runMopper(rc)

procedure runTower(rc):
    target <- searchWeakestEnemyRobot(rc)
    if target != null and rc.canAttack(target) then rc.attack(target)
    if rc.canUpgradeSelf(rc) then upgradeSelf(rc)
    unitType <- chooseSpawnType(rc)
    dir <- searchBuildDirection(rc, unitType)
    if dir != null then rc.buildRobot(unitType, dir)

procedure runSoldier(rc):
    if needPaint(rc) then
        refillPaint(rc)
        return
    ruin <- searchNearestOpenRuin(rc)

```

```

    if ruin != null then
        handleRuin(rc, ruin, DEFENSE_TOWER)
        return
    enemy <- searchWeakestEnemyRobot(rc)
    if enemy != null then
        rc.attack(enemy)
        return
    tower <- searchNearestAllyTower(rc)
    if tower != null then
        defendTower(rc, tower)
        return
    wanderAndPaint(rc)

procedure runMopper(rc):
    if needPaintMopper(rc) then
        refillPaint(rc)
        return
    enemy <- searchWeakestEnemyRobot(rc)
    if enemy != null then
        rc.attack(enemy)
        return
    dirty <- searchNearestEnemyPaint(rc)
    if dirty != null then
        rc.attack(dirty)
        return
    tower <- searchNearestAllyTower(rc)
    if tower != null then
        moveToward(rc, tower)

procedure runSplasher(rc):
    if needPaintSplasher(rc) then
        refillPaint(rc)
        return
    target <- searchBestSplashCenter(rc)
    if target != null then
        rc.attack(target)
        return
    tower <- searchNearestAllyTower(rc)
    if tower != null then moveToward(rc, tower)

```

4.2 Implementasi Solusi Akhir

4.2.1 Impressive General

```

package ImpressiveGeneral;

import java.util.Random;

import battlecode.common.*;

/**
 * RobotPlayer is the class that describes your main robot strategy.
 * The run() method inside this class is like your main function: this is what we'll call
 * once your robot
 * is created!
 */
public class RobotPlayer {
    /**
     * We will use this variable to count the number of turns this robot has been alive.

```

```

* You can use static variables like this to save any information you want. Keep in mind
  that even though
* these variables are static, in Battlecode they aren't actually shared between your
  robots.
*/
static int turnCount = 0;
static int uselessCount = 0;
static UnitType robotToBuild = UnitType.SOLDIER;
static UnitType towerToBuild = UnitType.LEVEL_ONE_PAINT_TOWER;
static MapLocation lastLoc = null;

/**
 * A random number generator.
 * We will use this RNG to make some random moves. The Random class is provided by the
   java.util.Random
 * import at the top of this file. Here, we *seed* the RNG with a constant number (6147);
   this makes sure
 * we get the same sequence of numbers every time this code is run. This is very useful
   for debugging!
 */
static final Random rng = new Random(6147);

/** Array containing all the possible movement directions. */
static final Direction[] directions = {
    Direction.NORTH,
    Direction.NORTHEAST,
    Direction.EAST,
    Direction.SOUTHEAST,
    Direction.SOUTH,
    Direction.SOUTHWEST,
    Direction.WEST,
    Direction.NORTHWEST,
};

/**
 * run() is the method that is called when a robot is instantiated in the Battlecode
   world.
 * It is like the main function for your robot. If this method returns, the robot dies!
 *
 * @param rc The RobotController object. You use it to perform actions from this robot,
   and to get
 *
 *       information on its current status. Essentially your portal to interacting
   with the world.
 */
@SuppressWarnings("unused")
public static void run(RobotController rc) throws GameActionException {
    // Hello world! Standard output is very useful for debugging.
    // Everything you say here will be directly viewable in your terminal when you run a
    match!
    // You can also use indicators to save debug notes in replays.

    while (true) {
        // This code runs during the entire lifespan of the robot, which is why it is in
        an infinite
        // loop. If we ever leave this loop and return from run(), the robot dies! At the
        end of the
        // loop, we call Clock.yield(), signifying that we've done everything we want to
        do.
        turnCount += 1; // We have now been alive for one more turn!

        if(rc.getType().isRobotType()) {
            if(turnCount > 1000) rc.disintegrate();

```

```

    }

    // Try/catch blocks stop unhandled exceptions, which cause your robot to explode.
    try {
        // The same run() function is called for every robot on your team, even if
        // they are
        // different types. Here, we separate the control depending on the UnitType,
        // so we can
        // use different strategies on different robots. If you wish, you are free to
        // rewrite
        // this into a different control structure!
        switch (rc.getType()){
            case SOLDIER: runSoldier(rc); break;
            case MOPPER: runMopper(rc); break;
            case SPLASHER: runSplasher(rc); break;
            default: runTower(rc); break;
        }

        if(rc.getActionCooldownTurns() < 10) uselessCount++;
        else if(uselessCount > 0) uselessCount--;
    }
    catch (GameActionException e) {
        // Oh no! It looks like we did something illegal in the Battlecode world. You
        // should
        // handle GameActionExceptions judiciously, in case unexpected events occur in
        // the game
        // world. Remember, uncaught exceptions cause your robot to explode!
        System.out.println("GameActionException");
        e.printStackTrace();

    } catch (Exception e) {
        // Oh no! It looks like our code tried to do something bad. This isn't a
        // GameActionException, so it's more likely to be a bug in our code.
        System.out.println("Exception");
        e.printStackTrace();

    } finally {
        // Signify we've done everything we want to do, thereby ending our turn.
        // This will make our code wait until the next turn, and then perform this
        // loop again.
        Clock.yield();
    }
    // End of loop: go back to the top. Clock.yield() has ended, so it's time for
    // another turn!
}

// Your code should never reach here (unless it's intentional)! Self-destruction
// imminent...
}

/**
 * Run a single turn for towers.
 * This code is wrapped inside the infinite loop in run(), so it is called once per turn.
 */
public static void runTower(RobotController rc) throws GameActionException{
    boolean enemyExist = false;
    for(MapInfo tile : rc.senseNearbyMapInfos()) {
        if(tile.isWall()) continue;
        if(tile.getPaint().isEnemy()) enemyExist = true;
    }
    enemyExist = enemyExist || (rc.senseNearbyRobots(-1, rc.getTeam().opponent()).length
    > 0);
}

```



```

if(rc.senseNearbyRobots().length < rc.getRoundNum() / 500 + 2 && !enemyExist)
    robotToBuild = UnitType.SOLDIER;
else {
    if(rc.getRoundNum() % 4 == 0) {
        robotToBuild = UnitType.MOPPER;
    }
    else {
        robotToBuild = UnitType.SPLASHER;
    }
}

// Pick a direction to build in.
Direction dir = bestDirection(rc);
MapLocation nextLoc = rc.getLocation().add(dir);

if ((enemyExist && rc.getPaint() > UnitType.SPLASHER.paintCost) ||
    (rc.canBuildRobot(robotToBuild, nextLoc) && rc.getChips() > 1000 +
     UnitType.SPLASHER.moneyCost && rc.getPaint() > UnitType.SPLASHER.paintCost)){
    rc.buildRobot(robotToBuild, nextLoc);
}

// Attack other bots
RobotInfo[] nearbyRobots = rc.senseNearbyRobots(-1, rc.getTeam().opponent());
for(RobotInfo robot : nearbyRobots) {
    if(!rc.canAttack(robot.getLocation())) break;
    rc.attack(robot.getLocation());
}

}

public static void runSoldier(RobotController rc) throws GameActionException{
    // Get paint if paint almost half empty
    if(uselessCount < 5) getPaint(rc);

    soldierObjective(rc);

    // Move and attack if no objective.
    if(rc.isMovementReady()) {
        Direction dir = bestDirection(rc);
        if (rc.canMove(dir)){
            lastLoc = rc.getLocation();
            rc.move(dir);
        }
    }

    // Try to paint beneath us as we walk to avoid paint penalties.
    // Avoiding wasting paint by re-painting our own tiles.
    if(rc.isActionReady()) {
        MapInfo currentTile = rc.senseMapInfo(rc.getLocation());
        if (rc.canAttack(rc.getLocation()) && (currentTile.getPaint() == PaintType.EMPTY
            || currentTile.getMark() != PaintType.EMPTY && currentTile.getMark() !=
            currentTile.getPaint())){
            rc.attack(rc.getLocation(), currentTile.getMark().isSecondary());
        }
        for (MapInfo nearbyTile : rc.senseNearbyMapInfos()){
            if (!rc.isActionReady()) break;
            if (nearbyTile.hasRuin() || nearbyTile.isWall()) continue;
            if (nearbyTile.getMark() != PaintType.EMPTY && nearbyTile.getMark() !=
                nearbyTile.getPaint() || nearbyTile.getPaint() == PaintType.EMPTY){
                if (rc.canAttack(nearbyTile.getMapLocation())) {
                    rc.attack(nearbyTile.getMapLocation(),
                        nearbyTile.getMark().isSecondary());
                }
            }
        }
    }
}

```

```

    }
    }
}

public static void runMopper(RobotController rc) throws GameActionException{
    // Get paint if paint almost half empty
    getPaint(rc);

    mopperObjective(rc);

    // Move and attack.
    if(!rc.isActionReady() && !rc.isMovementReady()) return;
    Direction nextDir = bestDirection(rc);
    MapLocation nextLoc = rc.getLocation().add(nextDir);
    if(!rc.senseMapInfo(nextLoc).getPaint().isEnemy()) {
        forceMove(nextLoc, rc);
    }

    MapLocation currentLoc = rc.getLocation();
    MapInfo currentTile = rc.senseMapInfo(currentLoc);

    if (rc.canAttack(currentLoc) && currentTile.getPaint().isEnemy()){
        rc.attack(currentLoc);
    }
    for (MapInfo nearbyTile : rc.senseNearbyMapInfos(2)){
        if (!rc.isActionReady()) break;
        if (nearbyTile.hasRuin() || nearbyTile.isWall()) continue;
        if (nearbyTile.getPaint().isEnemy()){
            if (rc.canAttack(nearbyTile.getMapLocation())) {
                rc.attack(nearbyTile.getMapLocation());
            }
        }
    }
}

public static void runSplasher(RobotController rc) throws GameActionException{
    // Get paint if paint almost half empty
    getPaint(rc);

    // Move and attack
    Direction dir = bestDirection(rc);
    MapLocation nextLoc = rc.getLocation().add(dir);
    if (rc.canMove(dir)){
        lastLoc = rc.getLocation();
        rc.move(dir);
    }
    if (rc.canAttack(nextLoc)){
        for (MapInfo nearbyTile : rc.senseNearbyMapInfos(4)){
            if (!rc.isActionReady()) break;
            if (nearbyTile.hasRuin() || nearbyTile.isWall()) continue;
            if (nearbyTile.getPaint().isEnemy() || nearbyTile.getPaint() ==
                PaintType.EMPTY){
                if (rc.canAttack(nearbyTile.getMapLocation())) {
                    rc.attack(nearbyTile.getMapLocation(),
                        nearbyTile.getPaint().isSecondary());
                }
            }
        }
    }
}
}

```

```

// Objective: build and upgrade towers
public static void soldierObjective(RobotController rc) throws GameActionException {
    if(rc.getNumberTowers() < 3 + rc.getRoundNum() / 20 && rc.getNumberTowers() < 5 &&
        rc.getNumberTowers() > 2) towerToBuild = UnitType.LEVEL_ONE_MONEY_TOWER;
    else towerToBuild = UnitType.LEVEL_ONE_PAINT_TOWER;

    // Sense information about all visible nearby tiles.
    MapInfo[] nearbyTiles = rc.senseNearbyMapInfos();
    // Search for a nearby ruin to complete.
    MapInfo curRuin = null;
    int curDistance = Integer.MAX_VALUE;
    for (MapInfo tile : nearbyTiles){
        if (tile.hasRuin() && !isTower(tile, rc)){
            int distance = rc.getLocation().distanceSquaredTo(tile.getMapLocation());
            if(distance < curDistance) {
                curRuin = tile;
                curDistance = distance;
            }
        }
        else if(isTower(tile, rc)) {
            if(rc.senseRobotAtLocation(tile.getMapLocation()).getType() ==
                UnitType.LEVEL_ONE_PAINT_TOWER)
                towerToBuild = UnitType.LEVEL_ONE_MONEY_TOWER;
        }
    }
    if (curRuin != null){
        MapLocation targetLoc = curRuin.getMapLocation();
        if(!isPatternFull(curRuin, rc)) {
            Direction dir = rc.getLocation().directionTo(targetLoc);
            forceMove(targetLoc, rc);
            // Mark the pattern we need to draw to build a tower here if we haven't
            // already.
            MapLocation shouldBeMarked = curRuin.getMapLocation().subtract(dir);
            if (rc.senseMapInfo(shouldBeMarked).getMark() == PaintType.EMPTY &&
                rc.canMarkTowerPattern(towerToBuild, targetLoc)){
                rc.markTowerPattern(towerToBuild, targetLoc);
            }
            // Fill in any spots in the pattern with the appropriate paint.
            for (MapInfo patternTile : rc.senseNearbyMapInfos(targetLoc, 8)){
                if (!rc.isActionReady()) break;
                if (patternTile.hasRuin() || patternTile.isWall()) continue;
                if (patternTile.getMark() != patternTile.getPaint() &&
                    patternTile.getMark() != PaintType.EMPTY &&
                    !patternTile.getPaint().isEnemy()){
                    if (rc.canAttack(patternTile.getMapLocation())) {
                        rc.attack(patternTile.getMapLocation(),
                            patternTile.getMark().isSecondary());
                    }
                }
            }
        }
    }

    // Complete the ruin if we can.
    if (rc.canCompleteTowerPattern(UnitType.LEVEL_ONE_MONEY_TOWER, targetLoc)){
        rc.completeTowerPattern(UnitType.LEVEL_ONE_MONEY_TOWER, targetLoc);
        rc.setTimelineMarker("Tower built", 0, 255, 0);
    }
    else if (rc.canCompleteTowerPattern(UnitType.LEVEL_ONE_PAINT_TOWER, targetLoc)){
        rc.completeTowerPattern(UnitType.LEVEL_ONE_PAINT_TOWER, targetLoc);
        rc.setTimelineMarker("Tower built", 0, 255, 0);
    }
}

```

```

    }

    RobotInfo[] robotNearby = rc.senseNearbyRobots(-1, rc.getTeam());
    for(RobotInfo robot: robotNearby) {
        if(robot.getType().isTowerType()) {
            MapLocation targetLoc = robot.getLocation();
            int threshold;
            if(robot.getType() == UnitType.LEVEL_ONE_PAINT_TOWER || robot.getType() ==
                UnitType.LEVEL_ONE_MONEY_TOWER || robot.getType() ==
                UnitType.LEVEL_ONE_DEFENSE_TOWER)
                threshold = 3500 + UnitType.SPLASHER.moneyCost * rc.getNumberTowers();
            else threshold = 6000 + UnitType.SPLASHER.moneyCost * rc.getNumberTowers();

            // Upgrade tower if can
            if(rc.canUpgradeTower(targetLoc) && rc.getChips() > threshold) {
                rc.upgradeTower(targetLoc);
                rc.setTimelineMarker("Tower upgraded at: " + targetLoc, 255, 255, 0);
                break;
            }
        }
    }
}

// Objective: kill oponent's robots and give paint (to robots other than soldiers)
public static void mopperObjective(RobotController rc) throws GameActionException {
    if(!rc.isActionReady()) return;

    RobotInfo[] nearbyEnemy = rc.senseNearbyRobots(-1, rc.getTeam().opponent());
    RobotInfo target = null;
    int targetDist = Integer.MAX_VALUE;
    for(RobotInfo robot : nearbyEnemy) {
        int dist = rc.getLocation().distanceSquaredTo(robot.getLocation());
        if(dist < targetDist) {
            target = robot;
            targetDist = dist;
        }
    }
    if(target != null) {
        MapLocation targetLoc = target.getLocation();
        if(rc.getLocation().distanceSquaredTo(targetLoc) > 2) {
            forceMove(targetLoc, rc);
        }

        Direction targetDir = rc.getLocation().directionTo(targetLoc);
        if(rc.canMopSwing(targetDir) && rc.getLocation().distanceSquaredTo(targetLoc) < 2)
        {
            rc.mopSwing(targetDir);
        }
    }
}

RobotInfo[] nearbyFriend = rc.senseNearbyRobots(-1, rc.getTeam());
target = null;
targetDist = Integer.MAX_VALUE;
for(RobotInfo robot : nearbyFriend) {
    if(robot.getType() == UnitType.MOPPER || robot.getType() == UnitType.SOLDIER ||
        robot.getType() == UnitType.LEVEL_ONE_PAINT_TOWER || robot.getType() ==
        UnitType.LEVEL_TWO_PAINT_TOWER || robot.getType() ==
        UnitType.LEVEL_THREE_PAINT_TOWER)
        continue;
    int dist = rc.getLocation().distanceSquaredTo(robot.getLocation());
    if(dist < targetDist) {
        target = robot;
    }
}

```

```

        targetDist = dist;
    }
}
if(target != null) {
    MapLocation targetLoc = target.getLocation();
    if(rc.getLocation().distanceSquaredTo(targetLoc) > 2) {
        forceMove(targetLoc, rc);
    }

    int giveAmount = rc.getPaint();
    if(target.getType().paintCapacity - target.getPaintAmount() < giveAmount)
        giveAmount = target.getType().paintCapacity - target.getPaintAmount();

    if(rc.canTransferPaint(targetLoc, giveAmount)) {
        rc.transferPaint(targetLoc, giveAmount);
    }
}
}

// Objective: paint as much enemy/empty tiles as possible
public static void splasherObjective(RobotController rc) throws GameActionException {
    if(!rc.isActionReady() && !rc.isMovementReady()) return;

    if(rc.isMovementReady()) {
        Direction dir = bestDirection(rc);
        MapLocation loc = rc.getLocation().add(dir);
        forceMove(loc, rc);
    }

    Direction dir = bestDirection(rc);
    MapLocation bestLoc;
    if(dir == Direction.NORTH || dir == Direction.EAST || dir == Direction.SOUTH || dir
        == Direction.WEST)
        bestLoc = rc.getLocation().add(dir).add(dir);
    else bestLoc = rc.getLocation().add(dir);

    if(bestLoc != null && rc.canAttack(bestLoc)) {
        rc.attack(bestLoc, rc.senseMapInfo(bestLoc).getPaint().isSecondary());
    }
}

// Check if tile is tower or not
public static boolean isTower(MapInfo tile, RobotController rc) throws
    GameActionException{
    MapLocation loc = tile.getMapLocation();
    if (rc.canSenseRobotAtLocation(loc)) {
        return rc.senseRobotAtLocation(loc).getType().isTowerType();
    }
    return false;
}

// Check if ruin's pattern completed
public static boolean isPatternFull(MapInfo tile, RobotController rc) throws
    GameActionException{
    MapLocation loc = tile.getMapLocation();
    MapInfo[] patternArea = rc.senseNearbyMapInfos(loc, 8);
    boolean markExist = false;
    for (MapInfo t : patternArea) {
        if(t.getMark() == PaintType.EMPTY) continue;
        else markExist = true;
        if(!(t.getMark() == t.getPaint() || t.getPaint().isEnemy())) return false;
    }
}

```

```

        return markExist;
    }

    public static void getPaint(RobotController rc) throws GameActionException{
        if(rc.getPaint() <= rc.getType().paintCapacity / 2 + rc.getType().attackCost) {
            MapInfo[] nearbyTile = rc.senseNearbyMapInfos(8);
            MapInfo towerTile = null;
            for(MapInfo tile : nearbyTile) {
                if(isTower(tile, rc)) {
                    UnitType towerType =
                        rc.senseRobotAtLocation(tile.getMapLocation()).getType();
                    if(towerType == UnitType.LEVEL_ONE_PAINT_TOWER || towerType ==
                        UnitType.LEVEL_TWO_PAINT_TOWER || towerType ==
                        UnitType.LEVEL_THREE_PAINT_TOWER) {
                        towerTile = tile;
                        break;
                    }
                }
            }
            if(towerTile != null) {
                MapLocation towerLoc = towerTile.getMapLocation();
                int paintNeeded = rc.getType().paintCapacity - rc.getPaint();
                if(paintNeeded > rc.senseRobotAtLocation(towerLoc).paintAmount)
                    paintNeeded = rc.senseRobotAtLocation(towerLoc).paintAmount;
                if(rc.canTransferPaint(towerLoc, -paintNeeded)) {
                    rc.transferPaint(towerLoc, -paintNeeded);
                }
            }
        }
    }

    public static void givePaint(RobotController rc) throws GameActionException {
        if(!rc.isMovementReady() && !rc.isActionReady()) return;

        RobotInfo[] nearbyRobots = rc.senseNearbyRobots(-1, rc.getTeam());
        RobotInfo target = null;
        int targetDist = Integer.MAX_VALUE;
        for(RobotInfo robot : nearbyRobots) {
            if(robot.getType() == UnitType.SOLDIER) continue;

            int missingPaint = robot.getType().paintCapacity - robot.getPaintAmount();
            if(missingPaint <= 0) continue;

            int dist = rc.getLocation().distanceSquaredTo(robot.getLocation());
            if(dist < targetDist) {
                target = robot;
                targetDist = dist;
            }
        }

        if(target != null) {
            MapLocation targetLoc = target.getLocation();
            if(rc.isMovementReady() && rc.getLocation().distanceSquaredTo(targetLoc) > 2) {
                forceMove(targetLoc, rc);
            }

            int availableToGive = rc.getPaint();
            int giveAmount = target.getType().paintCapacity - target.getPaintAmount();
            if(giveAmount > availableToGive) giveAmount = availableToGive;

            if(giveAmount > 0 && rc.canTransferPaint(targetLoc, giveAmount)) {
                rc.transferPaint(targetLoc, giveAmount);
            }
        }
    }

```

```

    }
}

// Find best direction for each robots
public static Direction bestDirection(RobotController rc) throws GameActionException{
    int[] paintScore = new int[8];
    MapInfo[] nearbyTiles = rc.senseNearbyMapInfos();

    for(MapInfo tile : nearbyTiles) {
        MapLocation loc = tile.getMapLocation();
        if(tile.isWall()) continue;
        Direction dirToTile = rc.getLocation().directionTo(loc);
        if(dirToTile == Direction.CENTER) continue;
        int index = dirToTile.ordinal();

        if(rc.getType() == UnitType.SOLDIER || (rc.getType().isTowerType() && robotToBuild
            == UnitType.SOLDIER)) {
            if(tile.getPaint() == PaintType.EMPTY) paintScore[index] += 100;
            else if(rc.canSenseRobotAtLocation(loc)) {
                paintScore[index] -= 200;
            }
            else if(tile.getPaint().isAlly()) paintScore[index] += 10;
            else if (tile.getPaint().isEnemy()) paintScore[index] += 5;
        }
        if(rc.getType() == UnitType.SPLASHER || rc.getType() == UnitType.MOPPER ||
            (rc.getType().isTowerType() && robotToBuild == UnitType.MOPPER) ||
            (rc.getType().isTowerType() && robotToBuild == UnitType.SPLASHER)) {
            if(rc.canSenseRobotAtLocation(loc)) {
                if(rc.senseRobotAtLocation(loc).getTeam() == rc.getTeam().opponent())
                    paintScore[index] += 100;
            }
            if(tile.getPaint().isEnemy()) paintScore[index] += 100;
            else if(rc.canSenseRobotAtLocation(loc)) {
                paintScore[index] -= 200;
            }
            else if(tile.getPaint() == PaintType.EMPTY) paintScore[index] += 10;
            else if (tile.getPaint().isAlly()) paintScore[index] += 5;
        }
    }

    int bestIndex = 0;
    Direction lastDir = Direction.CENTER;
    if(lastLoc != null) lastDir = rc.getLocation().directionTo(lastLoc);
    if(lastDir == directions[bestIndex]) bestIndex++;
    MapLocation bestLoc = rc.getLocation().add(directions[bestIndex]);
    for(int i = bestIndex + 1; i < 8; i++) {
        if(paintScore[i] >= paintScore[bestIndex] && lastDir != directions[i] &&
            (rc.canMove(directions[i]) || rc.canBuildRobot(robotToBuild, bestLoc))) {
            bestIndex = i;
            bestLoc = rc.getLocation().add(directions[bestIndex]);
        }
    }
    return directions[bestIndex];
}

// Ensure robot movement each turn (except when the robot's surrounded)
public static void forceMove(MapLocation loc, RobotController rc) throws
    GameActionException {
    if(!rc.isMovementReady()) return;
    Direction dir = rc.getLocation().directionTo(loc);
    for(int i = 0; i < 8 && !rc.canMove(dir); i++) {

```

```

        dir = dir.rotateRight();
    }
    if (rc.canMove(dir)) {
        rc.move(dir);
    }
}
}
}

```

4.2.2 Mr Capitalist

```

package MrCapitalist;

import battlecode.common.*;

public class RobotPlayer {

    static int turnCount = 0;

    static final Direction[] DIRS = {
        Direction.NORTH,
        Direction.NORTHEAST,
        Direction.EAST,
        Direction.SOUTHEAST,
        Direction.SOUTH,
        Direction.SOUTHWEST,
        Direction.WEST,
        Direction.NORTHWEST,
    };

    // Map Related
    static int mapW, mapH, myID, localSpawnCount;
    static MapLocation home, enemyBase, stickyRuin;
    static int laneY = -1;
    static final int LANE_H = 4;

    // SRP Related
    static int knownTowerCount = -1;
    static int lastTowerGainRound = 0;
    static final int SRPDelay = 20;

    @SuppressWarnings("unused")
    public static void run(RobotController rc) throws GameActionException {
        home = rc.getLocation();
        mapW = rc.getMapWidth();
        mapH = rc.getMapHeight();
        myID = rc.getID();
        enemyBase = new MapLocation(mapW - 1 - home.x, mapH - 1 - home.y);

        while (true) {
            turnCount++;
            try {
                switch (rc.getType()) {
                    case SOLDIER:
                        runSoldier(rc);
                        break;
                    case MOPPER:
                        runMopper(rc);
                        break;
                    case SPLASHER:
                        runSplasher(rc);

```



```

        break;
    default:
        runTower(rc);
        break;
    }
} catch (Exception e) {
    e.printStackTrace();
} finally {
    Clock.yield();
}
}

static void runTower(RobotController rc) throws GameActionException {
    RobotInfo weakest = null;
    for (RobotInfo e : rc.senseNearbyRobots(-1, rc.getTeam().opponent())) {
        if (!rc.canAttack(e.location))
            continue;
        if (weakest == null || e.health < weakest.health)
            weakest = e;
    }
    if (weakest != null)
        rc.attack(weakest.location);

    if (!rc.isActionReady())
        return;

    if (rc.canUpgradeTower(rc.getLocation())) {
        if ((rc.getRoundNum() > 150 && rc.getChips() > 2200) || (rc.getRoundNum() > 320 &&
            rc.getChips() > 1400)) {
            rc.upgradeTower(rc.getLocation());
            return;
        }
    }

    UnitType want = chooseSpawnType(rc);
    MapLocation spawn = null;
    for (Direction d : DIRS) {
        MapLocation loc = rc.getLocation().add(d);
        if (rc.canBuildRobot(want, loc)) {
            spawn = loc;
            break;
        }
    }
    if (spawn != null && rc.canBuildRobot(want, spawn)) {
        rc.buildRobot(want, spawn);
        localSpawnCount++;
    }
}

static UnitType chooseSpawnType(RobotController rc) {
    int round = rc.getRoundNum();
    int towers = rc.getNumberTowers();
    int chips = rc.getChips();

    if (round < 80 || towers < 3) {
        return (localSpawnCount % 8 == 7) ? UnitType.MOPPER : UnitType.SOLDIER;
    }

    if (chips < 1800) {
        return (localSpawnCount % 10 == 9) ? UnitType.MOPPER : UnitType.SOLDIER;
    }
}

```

```

    if (chips < 3500) {
        int slot = localSpawnCount % 10;
        if (slot == 8)
            return UnitType.MOPPER;
        if (slot == 9)
            return UnitType.SPLASHER;
        return UnitType.SOLDIER;
    }

    if (chips < 6000) {
        int slot = localSpawnCount % 8;
        if (slot <= 3)
            return UnitType.SPLASHER;
        if (slot == 4)
            return UnitType.MOPPER;
        return UnitType.SOLDIER;
    }

    int slot = localSpawnCount % 10;
    if (slot <= 5)
        return UnitType.SPLASHER;
    if (slot == 6)
        return UnitType.MOPPER;
    return UnitType.SOLDIER;
}

static void runSoldier(RobotController rc) throws GameActionException {
    MapLocation here = rc.getLocation();
    updateTowerProgress(rc);

    if (rc.getPaint() < (stickyRuin != null ? 35 : 20)) {
        refill(rc);
        return;
    }

    if (rc.isActionReady() && rc.canAttack(here) &&
        !rc.senseMapInfo(here).getPaint().isAlly()) {
        rc.attack(here);
    }

    if (stickyRuin != null && rc.canSenseRobotAtLocation(stickyRuin)) {
        RobotInfo ri = rc.senseRobotAtLocation(stickyRuin);
        if (ri != null && ri.getType().isTowerType())
            stickyRuin = null;
    }

    if (stickyRuin == null)
        stickyRuin = nearestOpenRuin(rc);

    if (stickyRuin != null) {
        doRuin(rc, stickyRuin);
        return;
    }

    if (rc.getRoundNum() > 300 && rc.getRoundNum() - lastTowerGainRound >= SRPDelay &&
        rc.getNumberTowers() >= 4
        && rc.getPaint() >= 80) {
        MapLocation srp = locateSRP(rc, here);
        if (srp != null) {
            doSRP(rc, srp);
            return;
        }
    }
}

```

```

    }
}

MapLocation goal = localExploreTarget(rc);
if (goal == null)
    goal = laneTarget(rc);
move(rc, goal);

if (rc.isActionReady())
    paintBest(rc);
}

static void doRuin(RobotController rc, MapLocation ruin) throws GameActionException {
    if (rc.canSenseRobotAtLocation(ruin)) {
        RobotInfo ri = rc.senseRobotAtLocation(ruin);
        if (ri != null && ri.getType().isTowerType()) {
            stickyRuin = null;
            return;
        }
    }

    UnitType towerType = chooseTowerType(rc);

    if (rc.getLocation().distanceSquaredTo(ruin) > 8) {
        move(rc, ruin);
        return;
    }

    boolean hasMarks = false;
    for (MapInfo mi : rc.senseNearbyMapInfos(ruin, 8)) {
        if (mi.getMark() != PaintType.EMPTY) {
            hasMarks = true;
            break;
        }
    }

    if (!hasMarks && rc.canMarkTowerPattern(towerType, ruin)) {
        rc.markTowerPattern(towerType, ruin);
    }

    if (rc.canCompleteTowerPattern(towerType, ruin)) {
        rc.completeTowerPattern(towerType, ruin);
        stickyRuin = null;
        lastTowerGainRound = rc.getRoundNum();
        return;
    }

    MapLocation target = null;
    boolean secondary = false;
    int bestDist = Integer.MAX_VALUE;

    for (MapInfo mi : rc.senseNearbyMapInfos(ruin, 8)) {
        if (mi.hasRuin() || mi.isWall() || mi.getMark() == PaintType.EMPTY)
            continue;

        boolean wrongPrimary = mi.getMark() == PaintType.ALLY_PRIMARY && mi.getPaint() !=
            PaintType.ALLY_PRIMARY;
        boolean wrongSecondary = mi.getMark() == PaintType.ALLY_SECONDARY && mi.getPaint()
            != PaintType.ALLY_SECONDARY;

        if (!wrongPrimary && !wrongSecondary)
            continue;
    }
}

```

```

        int d = rc.getLocation().distanceSquaredTo(mi.getMapLocation());
        if (d < bestDist) {
            bestDist = d;
            target = mi.getMapLocation();
            secondary = wrongSecondary;
        }
    }

    if (target != null) {
        if (rc.canAttack(target))
            rc.attack(target, secondary);
        else
            move(rc, target);
    } else {
        move(rc, ruin);
    }

    if (rc.canCompleteTowerPattern(towerType, ruin)) {
        rc.completeTowerPattern(towerType, ruin);
        stickyRuin = null;
        lastTowerGainRound = rc.getRoundNum();
    }
}

static UnitType chooseTowerType(RobotController rc) {
    int towers = rc.getNumberTowers();
    int i = towers % 3;
    if (i == 1 || i == 2)
        return UnitType.LEVEL_ONE_MONEY_TOWER;
    return UnitType.LEVEL_ONE_PAINT_TOWER;
}

/**
 * Run a single turn for a Mopper.
 * This code is wrapped inside the infinite loop in run(), so it is called once
 * per turn.
 */
static void runMopper(RobotController rc) throws GameActionException {
    MapLocation here = rc.getLocation();

    if (rc.getPaint() < 20) {
        refill(rc);
        return;
    }

    RobotInfo nearestEnemy = nearestEnemyUnit(rc);
    if (nearestEnemy != null) {
        Direction d = here.directionTo(nearestEnemy.location);
        if (rc.canMopSwing(d)) {
            rc.mopSwing(d);
            return;
        }
        if (rc.canAttack(nearestEnemy.location)) {
            rc.attack(nearestEnemy.location);
            return;
        }
        move(rc, nearestEnemy.location);
        return;
    }

    MapLocation dirty = nearestEnemyPaintLoc(rc);

```

```

        if (dirty != null) {
            if (rc.canAttack(dirty))
                rc.attack(dirty);
            else
                move(rc, dirty);
            return;
        }

        move(rc, laneTarget(rc));
    }

    /**
     * Run a single turn for a Splasher.
     * This code is wrapped inside the infinite loop in run(), so it is called once
     * per turn.
     */
    static void runSplasher(RobotController rc) throws GameActionException {
        if (rc.getPaint() < 45) {
            refill(rc);
            return;
        }

        MapLocation best = null;
        int bestScore = 4;

        for (MapInfo t : rc.senseNearbyMapInfos()) {
            MapLocation loc = t.getMapLocation();
            if (!rc.canAttack(loc))
                continue;

            int score = 0;
            for (MapInfo s : rc.senseNearbyMapInfos(loc, 2)) {
                PaintType p = s.getPaint();
                if (p.isEnemy())
                    score += 4;
                else if (p == PaintType.EMPTY)
                    score += 2;
                else
                    score -= 1;
            }

            if (score > bestScore) {
                bestScore = score;
                best = loc;
            }
        }

        if (best != null) {
            rc.attack(best);
            return;
        }

        move(rc, enemyBase);
    }

    static MapLocation nearestOpenRuin(RobotController rc) throws GameActionException {
        MapLocation best = null;
        int bestDist = Integer.MAX_VALUE;

        for (MapLocation ruin : rc.senseNearbyRuins(-1)) {
            if (rc.canSenseRobotAtLocation(ruin)) {
                RobotInfo ri = rc.senseRobotAtLocation(ruin);

```

```

        if (ri != null && ri.getType().isTowerType())
            continue;
    }
    int d = rc.getLocation().distanceSquaredTo(ruin);
    if (d < bestDist) {
        bestDist = d;
        best = ruin;
    }
}
return best;
}

static RobotInfo nearestEnemyUnit(RobotController rc) throws GameActionException {
    RobotInfo best = null;
    int bestDist = Integer.MAX_VALUE;

    for (RobotInfo e : rc.senseNearbyRobots(-1, rc.getTeam().opponent())) {
        if (e.getType().isTowerType())
            continue;
        int d = rc.getLocation().distanceSquaredTo(e.location);
        if (d < bestDist) {
            bestDist = d;
            best = e;
        }
    }
    return best;
}

static MapLocation nearestAllyTower(RobotController rc) throws GameActionException {
    MapLocation best = null;
    int bestDist = Integer.MAX_VALUE;

    for (RobotInfo r : rc.senseNearbyRobots(-1, rc.getTeam())) {
        if (!r.getType().isTowerType())
            continue;
        int d = rc.getLocation().distanceSquaredTo(r.location);
        if (d < bestDist) {
            bestDist = d;
            best = r.location;
        }
    }
    return best;
}

static MapLocation nearestEnemyPaintLoc(RobotController rc) throws GameActionException {
    MapLocation best = null;
    int bestDist = Integer.MAX_VALUE;

    for (MapInfo t : rc.senseNearbyMapInfos()) {
        if (!t.getPaint().isEnemy())
            continue;
        int d = rc.getLocation().distanceSquaredTo(t.getMapLocation());
        if (d < bestDist) {
            bestDist = d;
            best = t.getMapLocation();
        }
    }
    return best;
}

static void refill(RobotController rc) throws GameActionException {
    MapLocation tower = nearestAllyTower(rc);

```

```

    if (tower == null)
        tower = home;

    if (rc.getLocation().distanceSquaredTo(tower) <= 2 && rc.isActionReady()) {
        int need = rc.getType().paintCapacity - rc.getPaint();
        if (need > 0 && rc.canTransferPaint(tower, -need)) {
            rc.transferPaint(tower, -need);
            return;
        }
    }

    move(rc, tower);
}

static void paintBest(RobotController rc) throws GameActionException {
    if (!rc.isActionReady() || rc.getPaint() <= 20)
        return;

    MapLocation here = rc.getLocation();
    MapLocation best = null;
    int bestScore = Integer.MIN_VALUE;

    for (MapInfo t : rc.senseNearbyMapInfos()) {
        MapLocation loc = t.getMapLocation();
        if (!rc.canAttack(loc))
            continue;
        if (t.hasRuin() || t.isWall() || t.getPaint().isAlly())
            continue;

        int score = 0;
        if (t.getPaint().isEnemy())
            score += 30;
        else if (t.getPaint() == PaintType.EMPTY)
            score += 12;
        if (t.getMark() != PaintType.EMPTY)
            score += 8;
        score -= here.distanceSquaredTo(loc);

        if (score > bestScore) {
            bestScore = score;
            best = loc;
        }
    }

    if (best != null)
        rc.attack(best);
}

static MapLocation localExploreTarget(RobotController rc) throws GameActionException {
    MapLocation here = rc.getLocation();
    MapLocation bestEnemy = null, bestMarked = null, bestEmpty = null;
    int dEnemy = Integer.MAX_VALUE, dMarked = Integer.MAX_VALUE, dEmpty =
        Integer.MAX_VALUE;

    for (MapInfo tile : rc.senseNearbyMapInfos()) {
        MapLocation loc = tile.getMapLocation();
        if (tile.hasRuin() || tile.isWall() || !tile.isPassable())
            continue;

        int d = here.distanceSquaredTo(loc);

        if (tile.getPaint().isEnemy()) {

```

```

        if (d < dEnemy) {
            dEnemy = d;
            bestEnemy = loc;
        }
        continue;
    }

    if (tile.getPaint() == PaintType.EMPTY && tile.getMark() != PaintType.EMPTY) {
        if (d < dMarked) {
            dMarked = d;
            bestMarked = loc;
        }
        continue;
    }

    if (tile.getPaint() == PaintType.EMPTY && d < dEmpty) {
        dEmpty = d;
        bestEmpty = loc;
    }
}

if (bestEnemy != null)
    return bestEnemy;
if (bestMarked != null)
    return bestMarked;
return bestEmpty;
}

static void updateTowerProgress(RobotController rc) {
    int towers = rc.getNumberTowers();
    if (knownTowerCount == -1) {
        knownTowerCount = towers;
        lastTowerGainRound = rc.getRoundNum();
        return;
    }
    if (towers > knownTowerCount) {
        knownTowerCount = towers;
        lastTowerGainRound = rc.getRoundNum();
    }
}

static MapLocation locateSRP(RobotController rc, MapLocation here) throws
    GameActionException {
    MapLocation best = null;
    int bestDist = Integer.MAX_VALUE;

    for (int dx = -5; dx <= 5; dx++) {
        for (int dy = -5; dy <= 5; dy++) {
            int x = here.x + dx;
            int y = here.y + dy;

            if (x < 0 || x >= mapW || y < 0 || y >= mapH)
                continue;
            if (x % 4 != 2 || y % 4 != 2)
                continue;

            MapLocation c = new MapLocation(x, y);
            if (here.distanceSquaredTo(c) > 20)
                continue;

            boolean nearRuin = false;
            for (MapLocation ruin : rc.senseNearbyRuins(-1)) {

```



```

        if (c.distanceSquaredTo(ruin) <= 9) {
            nearRuin = true;
            break;
        }
    }
    if (nearRuin)
        continue;

    int dist = here.distanceSquaredTo(c);
    if (dist < bestDist) {
        bestDist = dist;
        best = c;
    }
}

return best;
}

static void doSRP(RobotController rc, MapLocation srp) throws GameActionException {
    if (rc.getLocation().distanceSquaredTo(srp) > 8) {
        move(rc, srp);
        return;
    }

    if (rc.canMarkResourcePattern(srp)) {
        rc.markResourcePattern(srp);
    }

    if (rc.canCompleteResourcePattern(srp)) {
        rc.completeResourcePattern(srp);
        lastTowerGainRound = rc.getRoundNum();
        return;
    }

    MapLocation target = null;
    boolean secondary = false;
    int bestDist = Integer.MAX_VALUE;

    for (MapInfo mi : rc.senseNearbyMapInfos(srp, 8)) {
        if (mi.getMark() == PaintType.EMPTY || mi.isWall() || mi.hasRuin())
            continue;

        boolean wrongPrimary = mi.getMark() == PaintType.ALLY_PRIMARY && mi.getPaint() !=
            PaintType.ALLY_PRIMARY;

        boolean wrongSecondary = mi.getMark() == PaintType.ALLY_SECONDARY
            && mi.getPaint() != PaintType.ALLY_SECONDARY;

        if (!wrongPrimary && !wrongSecondary)
            continue;

        int dist = rc.getLocation().distanceSquaredTo(mi.getMapLocation());
        if (dist < bestDist) {
            bestDist = dist;
            target = mi.getMapLocation();
            secondary = wrongSecondary;
        }
    }

    if (target != null) {
        if (rc.canAttack(target)) {

```

```

        rc.attack(target, secondary);
    } else {
        move(rc, target);
        return;
    }
}

if (rc.canCompleteResourcePattern(srp)) {
    rc.completeResourcePattern(srp);
    lastTowerGainRound = rc.getRoundNum();
}
}

static MapLocation laneTarget(RobotController rc) {
    if (laneY == -1) {
        int numLanes = Math.max(1, (mapH + LANE_H - 1) / LANE_H);
        int laneIdx = Math.floorMod(myID, numLanes);
        laneY = Math.min(laneIdx * LANE_H + LANE_H / 2, mapH - 1);
    }

    MapLocation here = rc.getLocation();
    if (Math.abs(here.y - laneY) > 1)
        return new MapLocation(here.x, laneY);

    return new MapLocation(enemyBase.x, laneY);
}

static void move(RobotController rc, MapLocation destination) throws GameActionException {
    if (!rc.isMovementReady() || destination == null)
        return;

    MapLocation here = rc.getLocation();
    Direction target = here.directionTo(destination);
    if (target == Direction.CENTER)
        target = DIRS[myID % 8];
    Direction best = null;
    int bestScore = Integer.MAX_VALUE;

    Direction d = target;
    for (int i = 0; i < 8; i++) {
        if (!rc.canMove(d))
        {
            d = d.rotateRight();
            continue;
        }

        MapLocation next = here.add(d);
        int score = next.distanceSquaredTo(destination) * 10;

        PaintType p = rc.senseMapInfo(next).getPaint();
        if (p.isEnemy())
            score += 15;
        else if (p.isAlly())
            score -= 4;

        if (score < bestScore) {
            bestScore = score;
            best = d;
        }

        d = d.rotateRight();
    }
}

```

```

        if (best != null)
            rc.move(best);
    }
}

```

4.2.3 Minister Of Defense

```

package MinisterOfDefense;

import battlecode.common.*;

import java.util.Arrays;
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Random;
import java.util.Set;

/**
 * RobotPlayer is the class that describes your main robot strategy.
 * The run() method inside this class is like your main function: this is what we'll call
 * once your robot
 * is created!
 */
public class RobotPlayer {
    /**
     * We will use this variable to count the number of turns this robot has been alive.
     * You can use static variables like this to save any information you want. Keep in mind
     * that even though
     * these variables are static, in Battlecode they aren't actually shared between your
     * robots.
     */
    static int turnCount = 0;

    /**
     * A random number generator.
     * We will use this RNG to make some random moves. The Random class is provided by the
     * java.util.Random
     * import at the top of this file. Here, we *seed* the RNG with a constant number (6147);
     * this makes sure
     * we get the same sequence of numbers every time this code is run. This is very useful
     * for debugging!
     */
    static final Random rng = new Random(6147);

    /** Array containing all the possible movement directions. */
    static final Direction[] directions = {
        Direction.NORTH,
        Direction.NORTHEAST,
        Direction.EAST,
        Direction.SOUTHEAST,
        Direction.SOUTH,
        Direction.SOUTHWEST,
        Direction.WEST,
        Direction.NORTHWEST,
    };

    // Simple defense-focused tuning knobs

```

```

static final int PAINT_LOW_SOLDIER = 60;
static final int PAINT_LOW_MOPPER = 40;
static final int UPGRADE_CHIPS_SOON = 1800;
static final int UPGRADE_ROUND_SOON = 150;
static final int UPGRADE_CHIPS_LATE = 1200;
static final int UPGRADE_ROUND_LATE = 280;
static int localSpawnCount = 0;

/**
 * run() is the method that is called when a robot is instantiated in the Battlecode
 * world.
 * It is like the main function for your robot. If this method returns, the robot dies!
 *
 * @param rc The RobotController object. You use it to perform actions from this robot,
 * and to get
 * information on its current status. Essentially your portal to interacting
 * with the world.
 */
@SuppressWarnings("unused")
public static void run(RobotController rc) throws GameActionException {
    // Hello world! Standard output is very useful for debugging.
    // Everything you say here will be directly viewable in your terminal when you run a
    // match!
    System.out.println("I'm alive");

    // You can also use indicators to save debug notes in replays.
    rc.setIndicatorString("Hello world!");

    while (true) {
        // This code runs during the entire lifespan of the robot, which is why it is in
        // an infinite
        // loop. If we ever leave this loop and return from run(), the robot dies! At the
        // end of the
        // loop, we call Clock.yield(), signifying that we've done everything we want to
        // do.

        turnCount += 1; // We have now been alive for one more turn!

        // Try/catch blocks stop unhandled exceptions, which cause your robot to explode.
        try {
            // The same run() function is called for every robot on your team, even if
            // they are
            // different types. Here, we separate the control depending on the UnitType,
            // so we can
            // use different strategies on different robots. If you wish, you are free to
            // rewrite
            // this into a different control structure!
            switch (rc.getType()){
                case SOLDIER: runSoldier(rc); break;
                case MOPPER: runMopper(rc); break;
                case SPLASHER: runSplasher(rc); break;
                default: runTower(rc); break;
            }
        }
        catch (GameActionException e) {
            // Oh no! It looks like we did something illegal in the Battlecode world. You
            // should
            // handle GameActionExceptions judiciously, in case unexpected events occur in
            // the game
            // world. Remember, uncaught exceptions cause your robot to explode!
            System.out.println("GameActionException");
            e.printStackTrace();
        }
    }
}

```

```

    } catch (Exception e) {
        // Oh no! It looks like our code tried to do something bad. This isn't a
        // GameActionException, so it's more likely to be a bug in our code.
        System.out.println("Exception");
        e.printStackTrace();
    } finally {
        // Signify we've done everything we want to do, thereby ending our turn.
        // This will make our code wait until the next turn, and then perform this
        // loop again.
        Clock.yield();
    }
    // End of loop: go back to the top. Clock.yield() has ended, so it's time for
    // another turn!
}

// Your code should never reach here (unless it's intentional)! Self-destruction
// imminent...
}

/**
 * Run a single turn for towers.
 * This code is wrapped inside the infinite loop in run(), so it is called once per turn.
 */
public static void runTower(RobotController rc) throws GameActionException{
    // Attack weakest enemy
    RobotInfo weakest = null;
    for (RobotInfo enemy : rc.senseNearbyRobots(-1, rc.getTeam().opponent())) {
        if (!rc.canAttack(enemy.getLocation())) continue;
        if (weakest == null || enemy.getHealth() < weakest.getHealth()) weakest = enemy;
    }
    if (weakest != null) rc.attack(weakest.getLocation());

    if (rc.isActionReady() && rc.canUpgradeTower(rc.getLocation())) {
        if ((rc.getRoundNum() > UPGRADE_ROUND_SOON && rc.getChips() > UPGRADE_CHIPS_SOON)
            || (rc.getRoundNum() > UPGRADE_ROUND_LATE && rc.getChips() >
                UPGRADE_CHIPS_LATE)) {
            rc.upgradeTower(rc.getLocation());
            return;
        }
    }

    UnitType want = chooseSpawnType(rc);
    MapLocation spawn = null;
    for (Direction d : directions) {
        MapLocation loc = rc.getLocation().add(d);
        if (rc.canBuildRobot(want, loc)) {
            spawn = loc;
            break;
        }
    }
    if (spawn != null && rc.canBuildRobot(want, spawn)) {
        rc.buildRobot(want, spawn);
        localSpawnCount++;
    }
}

private static UnitType chooseSpawnType(RobotController rc) {
    int chips = rc.getChips();
    int round = rc.getRoundNum();

```

```

// awal awal banyakin soldier dulu
if (round < 100 || chips < 1200) {
    return (localSpawnCount % 6 == 5) ? UnitType.MOPPER : UnitType.SOLDIER;
}

// mid game kasih splasher
if (chips < 2500) {
    int slot = localSpawnCount % 8;
    if (slot == 6) return UnitType.MOPPER;
    if (slot == 7) return UnitType.SPLASHER;
    return UnitType.SOLDIER;
}

// kalo kaya banyakin splasher
int slot = localSpawnCount % 10;
if (slot <= 4) return UnitType.SPLASHER;
if (slot == 5) return UnitType.MOPPER;
return UnitType.SOLDIER;
}

/**
 * Run a single turn for a Soldier.
 * This code is wrapped inside the infinite loop in run(), so it is called once per turn.
 */
public static void runSoldier(RobotController rc) throws GameActionException{
    MapLocation here = rc.getLocation();

    // Refill paint when low
    if (rc.getPaint() < PAINT_LOW_SOLDIER) {
        MapLocation tower = nearestAllyTower(rc);
        if (tower != null && rc.getLocation().distanceSquaredTo(tower) <= 2 &&
            rc.canTransferPaint(tower, -10)) {
            int need = rc.getType().paintCapacity - rc.getPaint();
            rc.transferPaint(tower, -need);
        } else {
            moveToward(rc, tower);
        }
        return;
    }

    // fokus bangun tower cuy
    MapLocation ruin = nearestOpenRuin(rc);
    if (ruin != null) {
        handleRuin(rc, ruin, UnitType.LEVEL_ONE_DEFENSE_TOWER);
        return;
    }

    // di awal mending banyakin keliling baru jaga tower
    if (rc.getRoundNum() < 120) {
        Direction dir = directions[rng.nextInt(directions.length)];
        if (rc.canMove(dir)) rc.move(dir);
        return;
    }

    // gelud lawan musuh
    RobotInfo targetEnemy = weakestEnemy(rc);
    if (targetEnemy != null) {
        if (rc.canAttack(targetEnemy.getLocation())) {
            rc.attack(targetEnemy.getLocation());
        } else {
            moveToward(rc, targetEnemy.getLocation());
        }
    }
}

```

```

        return;
    }

    //jaga tower as default
    MapLocation allyTower = nearestAllyTower(rc);
    if (allyTower != null) {
        if (here.distanceSquaredTo(allyTower) > 8) moveToward(rc, allyTower);
        paintPriorityTile(rc);
        return;
    }

}

/**
 * Run a single turn for a Mopper.
 * This code is wrapped inside the infinite loop in run(), so it is called once per turn.
 */
public static void runMopper(RobotController rc) throws GameActionException{
    // Refill paint
    if (rc.getPaint() < PAINT_LOW_MOPPER) {
        MapLocation tower = nearestAllyTower(rc);
        if (tower != null) moveToward(rc, tower);
        return;
    }

    // fokus jaga tower dan lawan musuh
    RobotInfo enemy = weakestEnemy(rc);
    if (enemy != null) {
        Direction d = rc.getLocation().directionTo(enemy.getLocation());
        if (rc.canMopSwing(d)) {
            rc.mopSwing(d);
        } else if (rc.canAttack(enemy.getLocation())) {
            rc.attack(enemy.getLocation());
        } else {
            moveToward(rc, enemy.getLocation());
        }
        return;
    }

    MapLocation enemyPaint = nearestEnemyPaint(rc);
    if (enemyPaint != null) {
        if (rc.canAttack(enemyPaint)) rc.attack(enemyPaint);
        else moveToward(rc, enemyPaint);
        return;
    }

    MapLocation tower = nearestAllyTower(rc);
    if (tower != null) moveToward(rc, tower);
}

public static void runSplasher(RobotController rc) throws GameActionException {
    if (rc.getPaint() < 80) {
        MapLocation tower = nearestAllyTower(rc);
        moveToward(rc, tower);
        return;
    }

    // Splash di tempat padet musuh
    MapLocation best = null;
    int bestScore = 0;

```

```

    for (MapInfo t : rc.senseNearbyMapInfos()) {
        MapLocation loc = t.getMapLocation();
        if (!rc.canAttack(loc)) continue;
        int score = 0;
        for (MapInfo a : rc.senseNearbyMapInfos(loc, 2)) {
            PaintType p = a.getPaint();
            if (p.isEnemy()) score += 4;
            else if (p == PaintType.EMPTY) score += 2;
            else score -= 1;
        }
        if (score > bestScore) {
            bestScore = score;
            best = loc;
        }
    }

    if (best != null) {
        rc.attack(best);
        return;
    }

    // jaga tower
    MapLocation tower = nearestAllyTower(rc);
    if (tower != null) moveToward(rc, tower);
}

private static void handleRuin(RobotController rc, MapLocation ruin, UnitType towerType)
    throws GameActionException {
    MapLocation here = rc.getLocation();
    if (here.distanceSquaredTo(ruin) > 8) {
        moveToward(rc, ruin);
        return;
    }

    if (rc.canMarkTowerPattern(towerType, ruin) && rc.senseMapInfo(ruin).getMark() ==
        PaintType.EMPTY) {
        rc.markTowerPattern(towerType, ruin);
    }

    for (MapInfo patternTile : rc.senseNearbyMapInfos(ruin, 8)) {
        if (patternTile.getMark() == PaintType.EMPTY) continue;
        if (patternTile.getMark() == patternTile.getPaint()) continue;
        boolean useSecondaryColor = patternTile.getMark() == PaintType.ALLY_SECONDARY;
        if (rc.canAttack(patternTile.getMapLocation())) {
            rc.attack(patternTile.getMapLocation(), useSecondaryColor);
            return;
        }
    }

    if (rc.canCompleteTowerPattern(towerType, ruin)) {
        rc.completeTowerPattern(towerType, ruin);
        rc.setTimelineMarker("Defense tower built", 0, 255, 0);
    }
}

private static void paintPriorityTile(RobotController rc) throws GameActionException {
    if (!rc.isActionReady()) return;
    MapLocation best = null;
    int bestScore = Integer.MIN_VALUE;
    MapLocation here = rc.getLocation();
    for (MapInfo t : rc.senseNearbyMapInfos()) {
        MapLocation loc = t.getMapLocation();

```



```

        if (!rc.canAttack(loc)) continue;
        if (t.hasRuin() || t.isWall()) continue;
        int score = 0;
        if (t.getPaint().isEnemy()) score += 20;
        else if (t.getPaint() == PaintType.EMPTY) score += 10;
        score -= here.distanceSquaredTo(loc);
        if (score > bestScore) {
            bestScore = score;
            best = loc;
        }
    }
    if (best != null) rc.attack(best);
}

private static RobotInfo weakestEnemy(RobotController rc) throws GameActionException {
    RobotInfo weakest = null;
    for (RobotInfo e : rc.senseNearbyRobots(-1, rc.getTeam().opponent())) {
        if (weakest == null || e.getHealth() < weakest.getHealth()) weakest = e;
    }
    return weakest;
}

private static MapLocation nearestAllyTower(RobotController rc) throws
    GameActionException {
    MapLocation best = null;
    int bestDist = Integer.MAX_VALUE;
    for (RobotInfo r : rc.senseNearbyRobots(-1, rc.getTeam())) {
        if (!r.getType().isTowerType()) continue;
        int d = rc.getLocation().distanceSquaredTo(r.getLocation());
        if (d < bestDist) {
            bestDist = d;
            best = r.getLocation();
        }
    }
    return best;
}

static MapLocation nearestOpenRuin(RobotController rc) throws GameActionException {
    MapLocation best = null;
    int bestDist = Integer.MAX_VALUE;

    for (MapLocation ruin : rc.senseNearbyRuins(-1)) {
        if (rc.canSenseRobotAtLocation(ruin)) {
            RobotInfo ri = rc.senseRobotAtLocation(ruin);
            if (ri != null && ri.getType().isTowerType())
                continue;
        }
        int d = rc.getLocation().distanceSquaredTo(ruin);
        if (d < bestDist) {
            bestDist = d;
            best = ruin;
        }
    }
    return best;
}

private static MapLocation nearestEnemyPaint(RobotController rc) throws
    GameActionException {
    MapLocation best = null;
    int bestDist = Integer.MAX_VALUE;
    for (MapInfo t : rc.senseNearbyMapInfos()) {
        if (!t.getPaint().isEnemy()) continue;

```

```

        int d = rc.getLocation().distanceSquaredTo(t.getMapLocation());
        if (d < bestDist) {
            bestDist = d;
            best = t.getMapLocation();
        }
    }
    return best;
}

private static void moveToward(RobotController rc, MapLocation target) throws
    GameActionException {
    if (target == null || !rc.isMovementReady()) return;
    Direction dir = rc.getLocation().directionTo(target);
    if (dir == Direction.CENTER) dir = directions[rng.nextInt(directions.length)];
    for (int i = 0; i < directions.length; i++) {
        Direction tryDir = dir;
        if (i > 0) tryDir = directions[(dir.ordinal() + i) % directions.length];
        if (rc.canMove(tryDir)) {
            rc.move(tryDir);
            return;
        }
    }
}

public static void updateEnemyRobots(RobotController rc) throws GameActionException{
    RobotInfo[] enemyRobots = rc.senseNearbyRobots(-1, rc.getTeam().opponent());
    if (enemyRobots.length != 0){
        rc.setIndicatorString("There are nearby enemy robots!");
        MapLocation[] enemyLocations = new MapLocation[enemyRobots.length];
        for (int i = 0; i < enemyRobots.length; i++){
            enemyLocations[i] = enemyRobots[i].getLocation();
        }
        RobotInfo[] allyRobots = rc.senseNearbyRobots(-1, rc.getTeam());
        if (rc.getRoundNum() % 20 == 0){
            for (RobotInfo ally : allyRobots){
                if (rc.canSendMessage(ally.location, enemyRobots.length)){
                    rc.sendMessage(ally.location, enemyRobots.length);
                }
            }
        }
    }
}
}

```

4.3 Pengujian dan Analisis

4.3.1 Mr Capitalist vs Minister Of Defense

Mr Capitalist, pada match ini berhasil memenangkan karena masih memiliki strategi coverage untuk mendapatkan ruin lebih sedangkan defense lebih mengedepankan defense dibanding explorasi. Sehingga banyak ruin yang tidak berhasil dicapai dan pada akhirnya akan tidak memiliki coverage untuk melawan.

4.3.2 Mr Capitalist vs Impressive General

General, pada match ini berhasil memenangkan karena apabila dibandingkan dengan Mr Capitalist, Impressive General memiliki system explorasi yang lebih mengedepankan coverage. Berbeda dengan capital yang memiliki system yang mengutamakan tower untuk di build apabila ada ruin yang dipenuhi, serta pembuatan SRP. Namun, bot Impressive General jauh lebih aggressive dalam explorasi awal dengan jumlah soldier yang lebih banyak. Mr Capitalist di sisi lain di sini mampu melawan



Figure 1: Mr Capitalist vs Minister Of Defense - Barcode

balik dikarenakan jumlah resource yang banyak dari tower dan SRP, walaupun jumlah bot yang spawn hampir sama antar kedua versi bot. Pada endgame pun jumlah chips yang dimiliki oleh Mr Capitalist jauh lebih banyak dibanding Impressive General



Figure 2: Mr Capitalist vs Impressive General - Oasis

4.3.3 Minister Of Defense vs Impressive General

Sekali lagi dikarenakan defense pada intinya hanya fokus pada mengamankan tower, bot ini tidak memiliki coverage dari eksplorasi lebih untuk mendapatkan advantage awal dibanding bot Impressive General



Figure 3: Minister Of Defense vs Impressive General - Alarm

5 Penutup

5.1 Kesimpulan

Dalam Tugas Besar 1 IF2211 Strategi Algoritma ini, kami berhasil mengimplementasikan algoritma greedy pada bot untuk memenangkan permainan battlecode. Bot utama bernama Impressive General berhasil diimplementasikan dengan pendekatan yang fokus untuk mengedepankan penyelesaian objektif khusus untuk tiap tipe robot.

Robot soldier memiliki objektif membangun tower, mopper menyerang robot lawan serta mendistribusikan cat, splasher mengecat sebanyak-banyaknya petak lawan ataupun petak kosong, serta tower membangun robot sebanyak-banyaknya dengan tipe robot sesuai dengan keadaan di sekitarnya.

Selain itu, kami juga berhasil membuat 2 bot alternatif, yaitu bot Mr Capitalist dengan pendekatan yang berfokus pada optimisasi resource. dan bot Minister Of Defense dengan pendekatan yang berfokus terhadap strategi defensif.

Memang banyak cara penggunaan *greedy algorithm* dalam menyelesaikan suatu permasalahan. Ada permasalahan yang bisa diselesaikan oleh banyak cara yang mengimplementasikan *greedy algorithm*, ada juga permasalahan yang sama sekali tidak bisa diselesaikan menggunakan algoritma tersebut. Walaupun begitu, *greedy algorithm* memberikan langkah awal dalam melihat suatu persoalan menjadi lebih mudah untuk dipahami dan diselesaikan.

5.2 Saran

Dalam pengerjaan Tugas Besar 1 IF2211 Strategi Algoritma ini, kami ingin berterima kasih kepada asisten yang telah membuat tugas besar yang menarik. Semoga setiap tahun pemilihan game untuk tugas besar ini bisa semakin menarik dan bermanfaat.

5.3 Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri

6 lampiran

github link: https://github.com/Arbane557/Tubes1_Stima_Impressed

References

- [1] R. Munir, “Algoritma Greedy (Bagian 1),” Bahan Kuliah IF2211 Strategi Algoritma, Program Studi Teknik Informatika, STEI, Institut Teknologi Bandung, 2026. [Online]. Available: [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/04-Algoritma-Greedy-\(2026\)-Bag1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/04-Algoritma-Greedy-(2026)-Bag1.pdf)